An Efficient New Approach Mean Based Sorting

D. Narasimha Raju
Department of Computer Science & Engineering
Shri Vishnu Engineering College for Women
Bhimavaram, India
dnraju@svecw.edu.in

***Abstract* - Data Structure is considered as one of the fundamental research area in computer science. Sorting is a foremost issue in data structure which arranges the data in ascending or descending order. Though several sorting algorithms are available, it is necessary to select the best sorting algorithm depends on the situation. These sorting technique are frequently used in a large variety of important applications. This paper presents a new sorting technique, "Mean Sort" which has exclusively been developed to sort the number of elements and also makes the data analysis and data processing very easy by taking the advantage of a computed mean. Like merge sort or quick sort, this algorithm works by single a divide-and-conquer strategy to divide a single unsorted array into two smaller sub-arrays. It uses mean to divide and sort them and time complexity is O(n*logn). This algorithm is described using C++. The proposed algorithm is analyzed using asymptotic analysis and also using machine-running time and compared with their basic sorting algorithms.**

Keywords: Data Structure, Divide and Conquer, Quick sort, Merge Sort, Mean Sort.

I. INTRODUCTION

There are many techniques for sorting. Implementation of particular sorting technique depends upon circumstances and differentiated by their computational complexity (behavior in best, average and worst) of element comparisons in terms of list size(n), number of swaps, stability, space requirements, recursion, whether or not they are a comparison sort [9]. The task of sorting is standard algorithmic benchmark, following are some sorting techniques: Bubble, Insertion, Selection, Shell, Bin, Quick, Merge, Heap [15]. Sorting techniques mainly depends on two parameters. First parameter is the execution time of program, which means time taken for execution of program. Second is the space, which means space taken by the program. Most simple sorting algorithms involve two steps which are compare two items and swap two items or copy one item. It continues executing over and over until the data is sorted [11].

The Mean Sort is a simple comparison sorting algorithm. It works by a divide and conquers strategy like Quick sort and Merge sort. A quick sort partition the array with the pivot element. All the smaller and greater elements are moved before and after the pivot respectively and then recursively sort the lesser and greater sublists. It has complexity of O(n*logn) [2]. A Merge sort divides the input array into two halves with computed mid position from the list and calls itself for the two halves and then joining the two sorted halves together. It has complexity of O(n*logn) [16].

The comparison sort algorithm sorts the data by comparing the data element with each other and selects the element location and swap on that location. The cost for swap operation is more critical than the comparison operation. Thus swap operation is completely eliminated by taking extra memory space.

II. MEAN SORT ALGORITHM

The basic concept is to compute mean. In this algorithm the computed mean may or not seems in the array list. The position of the mean in the list will be fixed after rearranging the elements. Array elements are rearranged such that all the elements less than the mean will be on left hand side and greater than will be on the right hand side. Later we will find out final location of the mean in the final sorted array by applying some method. Now divide the list into two parts, where the element is greater than mean. Thus, we apply this logic recursively on both the sides. This process continues until the list having only one value or set of duplicated values.

The Mean Sort algorithm does not require swap operation, but only comparisons operation. This algorithm is relatively better and faster than other sorting algorithms like Quick sort, Enhanced Selection sort (ESS), etc., when all or some of the elements get duplicated. For example when all the elements in the input array are same, requires very less number of comparison operations than required by quick and ESS. Its time complexity is O(n) otherwise O(n*logn).

The mean-sort algorithm works as follows:

- If there is one or less element in the array to be sorted, return immediately.
- Compute the mean from the array elements.
- Rearrange the elements in the array in such a way that all elements that are smaller than the mean move left to the mean and all the elements higher than the mean comes after it (equal values can go either way, in general we move to the left).
- Determine the position of mean in the array whether mean stands at the last position or not
  If position of mean is at the last then
  Set the starting position of the array to where mean stands
  Otherwise
  Recursively sort the two sub-arrays thus obtained. (One with sub-list of values smaller than that of the mean and the other having higher value elements).
- This process will continue till the list of elements exhausts or having only one.

978-1-4673-8507-7/15/$31.00 ©2015 IEEE

Note that when all the elements in the array list are same, then the mean could not split because mean itself stands at the last position. Then change the position of lower bound to where mean stands. It shows that the list has only one element since lower bound and upper bound, both pointing to the same element and continues to the next recursive call.

A. PSEUDOCODE

The following Mean Sort sorting algorithm makes uses a mean to divide the array into two sub-arrays and sort.

Mean Sort is a comparison sort. It is a divide and conquer algorithm, and uses recursion to sort the lists. It uses a mean chosen by the programmer, and passes through the sorting and on a certain condition, it sorts the data set.

In this pseudo code we use array subscript form where A[] is the array to be sort, mean is the variable for mean value obtained from the selected list and temp is a temporary pointer variable used to store the list temporarily. Count and sum variables are used to store count and sum of the elements in the list respectively. Initially these two variables are set to zero.

Mean Sort algorithm can be depicted as follows:

Call function MeanSort with Array A, Lower Bound lb, Upper Bound ub

Function MeanSort (A, lb, ub)

declare coumt, sum, lend, uend, pointer variable temp

1*: intialize count=0, sum=0, lend=lb, uend=ub*

2*: if the size of array >= 1 then*

3*: for i : lb to ub step 1 until i <= ub*

4*: compute sum = sum + A[i]*

5*: add 1 to count*

6*: end of loop*

7: *compute mean = sum / count*

8: for i: lb to ub step 1 until i <= ub

9: *if A[i] <= mean then*

10: *set temp[lend] = A[i]*

11: *add 1 to lend*

12: *else*

13: *set emp[uend] = A[i]*

14: *subtract 1 from uend*

15: *end if*

16: *end of loop*

17: *if lend-1 = ub then*

18: *set lb = ub*

19: *free(r)*

20: *else*

21: *for i: lb to lend step 1 until i<lend*

22: *set A[i] = temp[i]*

23: *end of loop*

24: *for i: ub to uend step -1 until i>uend*

25: *set A[i] = temp[i]*

26: e*nd of loop*

27: *free(r)*

28: *MeanSort (A, lb, lend-1)*

29: *MeanSort (A, uend+1, ub)*

30: *End if*

31: *End if*

32: *Stop*

B. WORKING OF THE ALGORITHM

Case 1: Suppose an array A contains 6 elements as follows:

27 10 36 18 25 45

Mean Sort works by finding the mean on the array elements from A[0] through A[5] ie., 27 to 45. Here the value of the mean is 26. Now rearrange the elements of an array A by moving all the elements into the other array temp in such a way that compare each and every element of an array A with the mean and place it from the left position of an array temp, if an element is less than mean, otherwise from the right position. Then copy all of the elements in an array temp into original array A. After rearrangement of this process recursively call Mean Sort function with first sub-array which consists of values smaller than the mean and then with second sub-array which contains the elements that are greater than the mean. They follow:

a) First subarray with elements smaller than mean, and their positions are from 0 to 2. Elements are as follow:
10 18 25

b) Second subarray with elements larger than mean, and the starting position of the array would become next to the last position where the mean had divided. The position is 3 extends upto position 5. Elements are follows:

45 36 25

Apply the same procedure to the both sublists until each sublist having only one element. The result would appear, at the end of procedure the elements are in sorted order as

10 18 25 27 36 45

The below Fig. 2b.1 illustrates it.

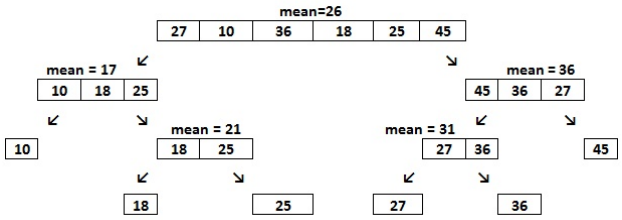


Fig. 2b.1. Flow of array elements when they are unique.

Case 2: Consider the same list which is above already had discussed, but with minor change take 10 instead of 25. When it encounters the subarray contains the same value 10 and also mean would be the same with 10. In this case mean cannot split the list. Therefore solution to this just leave the list and continue with the next recursive call ie., with the right subarray by making equal lower bound position with upper bound of the array. Same conveyed in the below Fig. 2b.2

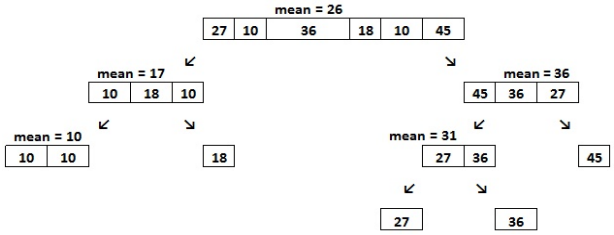


Fig. 2b.2. Flow of array elements when some duplicate element existed.

Case 3: Below Fig. 2b.3 depicts that array list has same value 10, and then the mean could not split because mean stands at the last position 5. It tells no more position in array and none of the value is different and does not require comparison. Therefore change the position of lower bound to where mean stands that seems the list has only one element, immediately terminate the process.

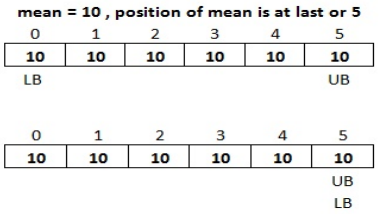


Fig 2b.3: Flow of array elements when all are same.

Note that temporary array is removed from memory at the end of each recursive call.

III. ANALYSIS

The **Time Complexity** of the proposed algorithm can be elaborated as follows:

- The worst–case: if the input array is already sorted or reversed or all elements in the list are unique, then it requires more number of comparisons and time complexity is O(n*logn), since it do not have inner loops throughout the program.
- The average-case: if the input array is having atleast one or more duplicate element, then it leaves the list position of lb set to ub. gives its best performance by reducing number of comparisons with O(n*logn) time complexity than the previous case.
- The best-case: if the elements in the input array all are same, after comparing all the elements in the list with mean, position of the mean is at last. No further subdivision is required and considered it as a final list. Hence the time complexity is O(n).

Space Complexity: In the first pass the size of the temporary array can be as the size of the original array. After dividing the array into two sublists, in the next pass the memory requirement is roughly less than the original size requires. This process will continue until the size of the array is one or zero. At the end of the each pass, algorithm calls function free (), to free up the temporary memory. Hence space complexity of the proposed algorithm is O (logn). Mean Sorting can be achieved without using extra memory, but its performance can be increased with the cost of some extra memory [13].

The mean of each original list or sub lists can be used as one of the comparing properties of statistics.

Here some of the applications of mean as follow:

- It helps to get the elements in one list that are smaller or equal to mean and greater than mean in other list.
- It helps teacher to see the average marks of the students.
- It is used to in factories, for the authorities to recognize whether the benefits of the workers are continued or not.
- It is also used to contrast the salaries of the workers.
- It is used to calculate the average speed of anything.
- It is also used by the government to find the income or expenses of any person.
- Using this family member could balance their expenses with their average income.

Table 1 shows the differences between criteria's of Mean Sort, Quick sort, Merge sort and Enhanced Selection sort. It shows that Mean Sort is relatively high when elements are same and relatively low when elements are unique, sorted, reversed as compared with above specified sorting techniques. The values for swap operation proven that nowhere it is required for Mean Sort.

TABLE I

Mean Sort VS Quick sort VS ESS VS Merge sort

Sort Name	No. of Inputs	Elements	No. of Comparisons	No. of Swaps
Quick sort	7	1 1 1 1 1 1 1	100	0
	6	26 10 36 18 25 45	59	5
ESS	7	1 1 1 1 1 1 1	45	0
	6	26 10 36 18 25 45	20	3
Merge Sort	7	1 1 1 1 1 1 1	37	0
	6	26 10 36 18 25 45	33	3

Mean Sort	7 6	1 1 1 1 1 1 1 26 10 36 18 25 45	23 85	0 0

IV. IMPLEMENTATION OF PROPOSED ALGORITHM

The proposed algorithm has been implemented in C++ programming language. The tool used for the implementation is NeuTroN DoS-C++ by Turbo C++ for windows 7.

The following screenshots shows the output of the execution of the code implemented. These are as follows:

a) When executes starts, the output console asks the user for entering the number of elements that are to be sorted. Let us suppose the entered number is 6 that are stored in a variable n which indicates the size of the array to be sorted.



Fig. 4a. Execution Output Console

b) The console window shows the current state of the execution. After giving the input of the number of elements, the user is required to input the elements of the array and stored in an array variable arr. The elements are as follows

27 10 36 18 25 45



Fig. 4b. Execution Output Console

c) Fig. 4c. Shows the elements entered by the user.

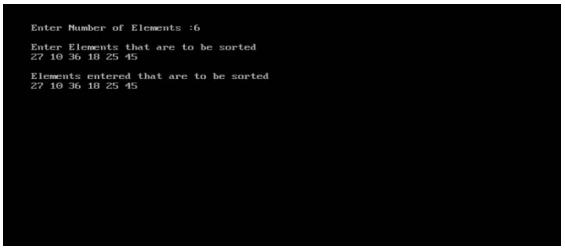


Fig. 4c.. Execution Output Console

d) The algorithm works in a two steps. First compute the mean. Secondly rearrange and divide the list based on the mean. At the end of the first pass the result shown in Fig. 4d as follows:

10 18 25 27 36 45



Fig. 4d. Execution Output Console

e) The above process continues until all elements are processed and the intermediate results of each pass and actual output is shown in Fig. 4e as follows.



Fig. 4e. Execution Output Console

V. CONCLUSION

In this paper, new sorting algorithm "Mean Sort" has been proposed. The developed sorting technique, "Mean Sort" is stable for sorting and also can take the advantage of mean which is obtained from each of the sub list. The performance of this algorithm is quite good and efficient since it requires less number of comparisons and has zero number of swap operations, but with a little bit overhead of computing mean of all the times. The complexity of the algorithm does not changes with the input cases, complexity remains constant O(n*logn) for the worst case as well as average case.

References

[1] "Analysis and Review of Sorting algorithms", Gaurav Kocher, Nikita Arawal

[2] Pankaj sareen, "Comparisions of Sorting algorithms (on the basis of Average Case)", Vol 3, Issue 3, Mar 13, IJARDSSE.

[3] Jehad Alnihoud and Rami Mansi, "An Enhancement of Major sorting algorithms", vol7, No.1, Jan 10, IJAJIT.

[4] Knuth D., The Art of computer Programming, Addison Wesley, 1998

[5] Cormen T., Leiserson C., Rivest R., and St ein C., Introduction to Algorithms, McGraw Hill, 2001.

[6] Levtin A., Introduction to Data Structures, Nyhoff Publishres, Amsterdam, 2005

[7] Shahzad B. and Afzal M., "Enhanced Shell Sorting Algorithm " computer journal of Enformatika, vol. 21 no. 6, pp. 66-70, 2007

[8] Aho. A., Hopcroft J., and Ullmn J., The Design and Analysis of Computer Algorithms, Addison Wesley, 1974.

[9] Quick sort is optimal by Robert Sedgewick and Jon Bently, Knuthfest, Stanford University, jan 2002.

[10] Sedgewick, Algorithms in c++, pp. 96-98, 102, ISBN 0-201-51059-6, Addison-Wesley, 1992.
[11] Data Structures with C – Seymour Lipschutz, Mc Graw Hill, special Indian edition.
[12] S. Lipschutz, "theory and Problems of data structure", McGraw Hill Book Company.
[13] Khairullah Md.,"Enhancing Worst Sorting Algorithms", July-2013, IJAST.
[14] Maurya V. N., Bathla R. K., Kaur Diwinder, Maurya Avadhesh, Gautam Ram Asrey, "An Alternate Efficient Sorting Algorithm Applicable for Classification of Versatile Data", April-2013 IJMMAC.
[15] Dr.V.N.Maurya, Dr. R.K.Bathla, Diwinder Kaur Arora, Er. Avadesh Kumar Maurya and Ram Asrey Gautam, "An Alternate Efficient Sorting Algorithm Applicable for Classification of Versatile Data", IJMMAC, Vol. 1, April 2013.
[16] Jariya Phongsai, Prof. Lawrence Muller, "Research Paper on Sorting Algorithms", Oct 26, 2009.



D. Narasimha Raju Author was born in Vijayawada, Andhra Pradesh, India in 1979. He received the Master degree in Computer Applications from Acharya Nagarjuna University and M.Tech. degree in Information Technology from the University of JNTUK, Kakinada in 2010.